4.6.2 Listing pages

A view doesn't have to take arguments to take advantage of the database. An index listing page could query all, or some, of the objects in the database and display them to the user. Each object in the result can then link to the corresponding detail page.

It is common practice to have the object page named in the singular and the listing page in the plural. The `musician()` view you wrote previously displays a single musician. The corresponding listing view would be called `musicians()` (plural). The database query inside the `musicians()` view looks for all the `Musician` objects in the database.

It is good practice to sort the contents of your listing pages. You do this by appending `.order_by()` to your query. This is called an *annotation*. Annotations interact with query results, changing their behavior or augmenting the information on the returned objects. The `.order_by()` annotation takes an argument: the name of the field to sort on. To sort musicians by last name, you use `.order_by("last_name")`.

The entire query result gets passed from the view to the template. The template then iterates over the result, displaying a URL for each object. In chapter 3, you learned about the `{% url %}` tag, which returns the URL corresponding to a named `path` for a registered route. Way back in listing 4.5, the `path` registering the `musician` view got named `musician`, so it can be rendered through the `{% url %}` tag. In addition to taking the name of a URL, the tag also takes any arguments. For the `musician` view, the argument is the musician's ID, which is accessed as an attribute of the object. Edit RiffMates/bands/views., and add the code for the `musicians` view, as seen in listing 4.8.

Listing 4.8 Index page for `Musician` objects

```
# RiffMates/bands/views.py
...
def musicians(request):
    data = {
        'musicians':Musician.objects.all().order_by('last_name'),
    }

    return render(request, "musicians.html", data)
```

Query all Musician objects, ordering by the last_name field.

You'll need to register the route for your new view. As you now have a URL file specific to the bands app, that is the best place to register the URL. This time the URL has no argument, so you're back to the simpler form. Edit RiffMates/bands/urls.py, and add the new `path` object like in the following listing.

Listing 4.9 Adding the `musicians` index view to the bands app URL file

```
# RiffMates/bands/urls.py

urlpatterns = [
...
    path('musicians/', views.musicians, name="musicians"),
]
```